

BeatPeeler: Source Separation using REpeating Pattern Extraction Technique (REPET)

ECE 420 Final Report

Sreeram Rave

University of Illinois Urbana-Champaign

{srave2}@illinois.edu

I. INTRODUCTION

Audio source separation, specifically isolating a vocal track from a background instrumental, has immense practical value in the music production industry. The primary product justification for *BeatPeeler* is to provide mobile users, such as DJs, producers, and karaoke enthusiasts, with an offline, on-device tool for remixing, sampling, and freestyling.

Currently, state-of-the-art separation relies on massive neural networks that require either cloud computing APIs or high-end desktop GPUs. This limits usability in mobile environments without internet access. The goal of this project is to bridge that gap by implementing a purely mathematical, unsupervised DSP algorithm that can run entirely on an Android tablet's local processor. By avoiding heavy machine learning models, the app maintains a small storage footprint while still delivering functional stem separation.

II. LITERATURE AND RESEARCH REVIEW

A. Primary Reference Description

The foundational algorithm for this project is the REpeating Pattern Extraction Technique (REPET), originally introduced by Zafar Rafii and Bryan Pardo [1]. REPET departs from complex, model-driven source separation by operating on a fundamental psychoacoustic and statistical assumption: in most popular music, the instrumental background is highly repetitive and cyclic, whereas the lead vocals are sparse, dynamic, and non-repeating. The algorithm isolates

the background by identifying the repeating beat period via autocorrelation, extracting a median-filtered background model, and subtracting it from the original mixture to reveal the vocals.

B. Other Approaches in Existing Literature

Audio source separation is a highly active research field. Existing applications and papers generally tackle the problem using one of two alternative methodologies, both of which possess critical flaws when deployed to an offline mobile environment:

- **Deep Learning (Spleeter, Demucs):** State-of-the-art commercial apps currently rely on Convolutional Neural Networks (CNNs) trained on thousands of hours of isolated audio stems. For context, models like Spleeter routinely achieve vocal Signal-to-Distortion Ratios (SDR) of 6 to 8 dB. While these models produce mathematically superior scores, they require massive memory footprints (hundreds of megabytes for the weights) and heavy GPU acceleration, making them largely incompatible with offline processing on standard Android tablets.
- **Traditional DSP (ICA and NMF):** Independent Component Analysis (ICA) is a classic separation technique, but it strictly requires multi-channel audio to mathematically unmix signals; it fails completely on mono mixdowns. Non-negative Matrix Factorization (NMF) can operate on mono signals, but it relies on iterative matrix optimization loops that are too computationally expensive for a mobile CPU to execute in a reasonable timeframe.

REPET was chosen over these approaches because it is entirely unsupervised. While unsupervised DSP methods typically yield lower separation quality (average SDR of 2 to 4 dB), this reduction is the necessary engineering trade-off for achieving a zero-dependency, mobile-executable pipeline that requires a predictable amount of matrix operations.

C. Engineering Contributions and "Secret Sauce"

While the core REPET theory was preserved, a direct port of the algorithm's matrix mathematics triggers Java Out-Of-Memory (OOM) exceptions on Android when processing full-length songs. To make the application viable for mobile hardware, *BeatPeeler* implements two unique architectural upgrades:

- 1) **Optimized Soft Wiener Masking:** The original REPET paper utilizes a harsh binary mask, which introduces severe "musical noise" artifacts (low SAR). *BeatPeeler* upgrades this to a

soft Wiener mask ($M = W^\beta / (V^\beta + \epsilon)$). Through objective testing, the attenuation severity was mathematically optimized to $\beta = 1.4$ to maximize the vocal SDR score while balancing algorithmic distortion.

- 2) **Native C++ Multithreading:** The 2D median filtering stage is highly computationally demanding. To prevent the Android OS from freezing, the Java frontend offloads the matrix arrays to a native C++ engine. A custom `parallelFor` routine was engineered to dynamically detect the tablet's CPU core count and distribute the frequency-bin median calculations across multiple threads, drastically reducing execution time and enabling on-device processing.

III. TECHNICAL DESCRIPTION

A. Algorithm Overview

The REPET algorithm is implemented in three primary stages: Period Detection, Background Modeling, and Masking/Extraction.

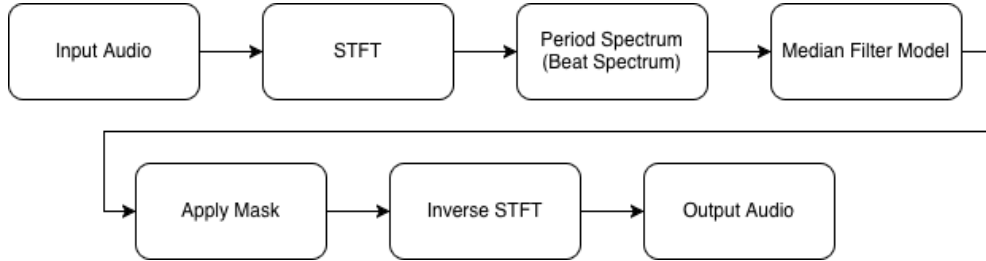


Fig. 1. Block diagram of the REPET processing pipeline.

Stage 1: Period Detection

The input audio $x[n]$ is transformed into the time-frequency domain using the Short-Time Fourier Transform (STFT), yielding the complex matrix $X(m, k)$. The magnitude spectrogram is defined as $V = |X|$. To emphasize percussive transients, the spectrogram is squared (V^2). The beat spectrum b is then derived by calculating the autocorrelation of V^2 across time frames for each frequency bin, and summing them. The repeating period, p , is the lag that maximizes the mean energy across its integer multiples.

Stage 2: Background Modeling

Once p is detected, the spectrogram V is segmented into r overlapping chunks of length p .

A repeating segment model S is derived by taking the element-wise median of these stacked segments:

$$S(m, k) = \text{median} (V_1(m, k), V_2(m, k), \dots, V_r(m, k)) \quad (1)$$

The median filter mathematically captures the consistent, repeating instrumental patterns while treating the sparse, varying vocal transients as statistical outliers, effectively discarding them.

Stage 3: Masking and Extraction

A full-length background model W is created by taking the element-wise minimum between S and the original spectrogram V . To extract the audio, a soft Wiener time-frequency mask M is calculated:

$$M = \frac{W^\beta}{V^\beta + \epsilon} \quad (2)$$

where β controls the severity of the attenuation and ϵ prevents division by zero. The mask M is applied to the complex STFT to isolate the background, and the foreground is derived via subtraction. Finally, an Inverse STFT (ISTFT) reconstructs the time-domain audio.

B. Modifications for Mobile (Windowing)

A direct port of REPET requires massive complex matrix allocations. Processing a 3-minute song simultaneously triggers Java Out-Of-Memory (OOM) exceptions on Android. To resolve this, the algorithm was modified to process audio in localized 10-second blocks with a 75% overlap. A 75% overlap was specifically chosen because it satisfies the Constant Overlap-Add (COLA) criterion for a Hann window, ensuring perfect amplitude reconstruction without modulation artifacts.

```
function process_full_audio(audio, window_sec, overlap, beta):
    window_samples = window_sec * fs
    hop_samples = window_samples * (1 - overlap)
    fade_window = hanning(window_samples)

    background_out = array_of_zeros(length(audio))
    foreground_out = array_of_zeros(length(audio))

    for each 10-second window in audio:
        # run repet on isolated chunk
        bg_chunk, fg_chunk, period = repet_separation(window, beta)
```

```

# apply hann window for smooth fading
bg_chunk = bg_chunk * fade_window
fg_chunk = fg_chunk * fade_window

# overlap and add into final arrays
background_out[start:end] += bg_chunk
foreground_out[start:end] += fg_chunk

normalize_by_overlap(background_out, foreground_out)
return background_out, foreground_out

```

Listing 1. Pseudocode for Windowed Overlap-Add REPET

IV. APP IN ACTION (SCREENSHOTS)

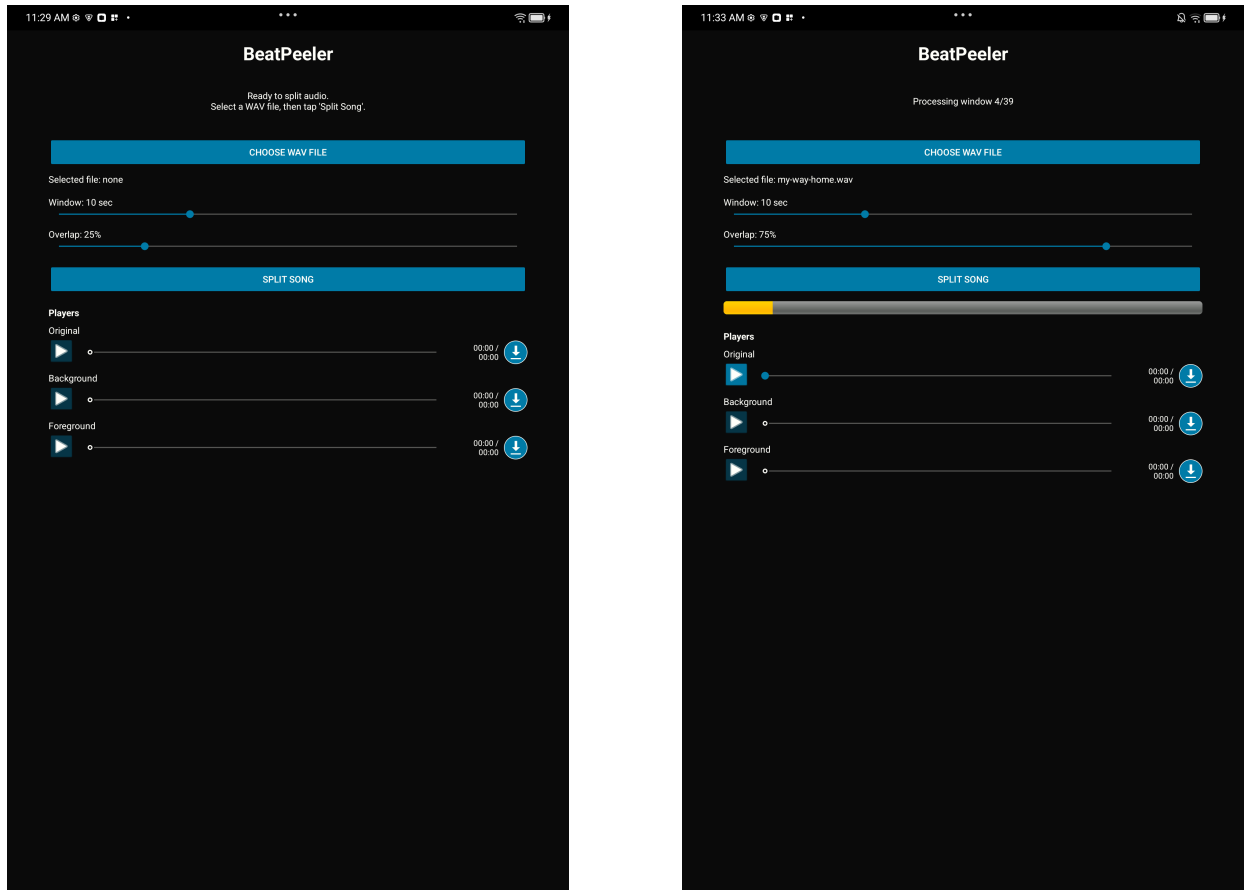


Fig. 2. Step-by-step UI Flow. **Left:** The Home screen where users import a local .wav file and adjust the sliding Window/Overlap parameters. **Right:** The Processing and Playback screen, displaying a responsive progress bar that accurately tracks the native C++ thread execution, followed by independent playback controls for the separated tracks.

V. FINAL RESULTS AND EVALUATION

A. Evaluation Methodology

Performance was measured objectively using the BSS Eval suite via Python’s `mir_eval` library. For ground-truth validation, state-of-the-art (SOTA) AI stem splitters were utilized to generate the reference foreground and background tracks. While official studio instrumentals are traditionally preferred for mathematical purity, they are rarely available for the diverse range of modern tracks required for testing.

Limitation Acknowledgment: It must be explicitly acknowledged that using AI splits as a ground truth is a limitation. Because our reference relies on SOTA AI outputs rather than pure studio stems, our SDR scores mathematically measure REPET’s adherence to the AI’s separation rather than the true source signals. Consequently, our metrics cannot be directly compared to published REPET benchmarks.

B. Parameter Tuning and Genre Variance (The β Sweep)

The β parameter in the Wiener mask dictates the strictness of the frequency separation. To isolate the mathematical impact of the mask, the SDR for each track was normalized to its baseline performance at $\beta = 1.0$ (ΔSDR).

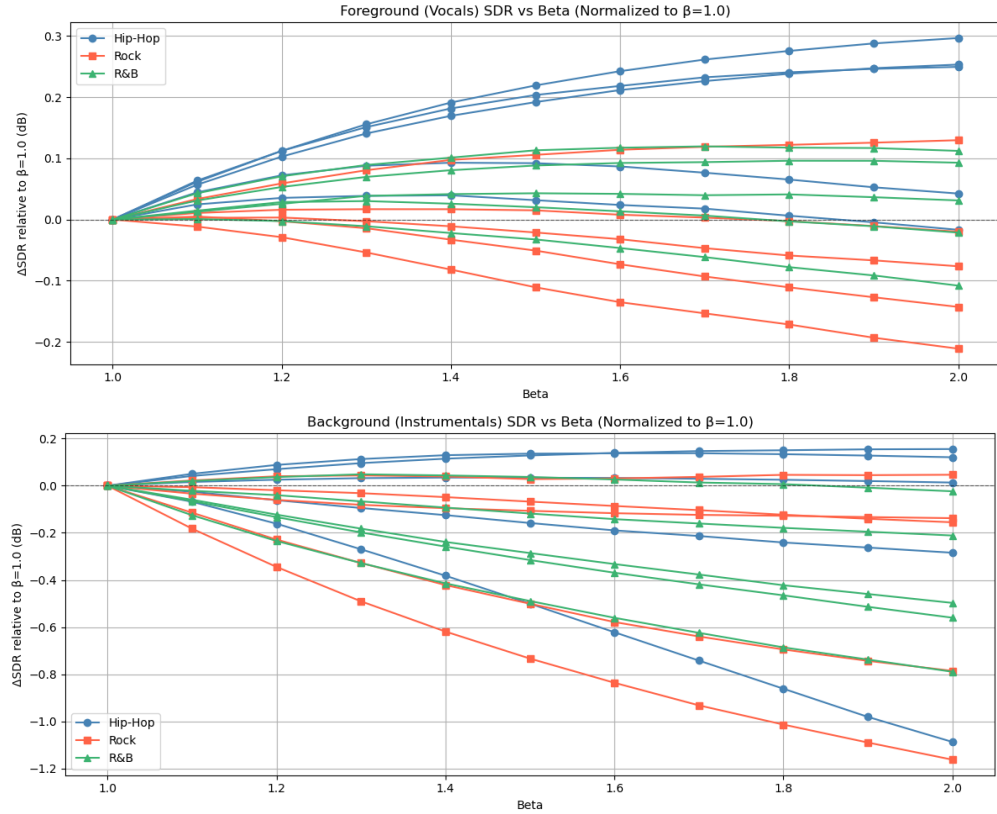


Fig. 3. Normalized Δ SDR vs. Beta (β) parameter sweep grouped by genre. Top: Foreground vocals exhibit high variance, with sparse Hip-Hop benefiting from harder masks. Bottom: Background instrumentals uniformly degrade as mask severity increases.

As illustrated in Figure 3, optimal mask severity is genre-dependent. Sparse, quantized Hip-Hop tracks benefit immensely from a harder mask ($\beta \rightarrow 2.0$), gaining up to +0.3 dB in SDR. Conversely, Rock and R&B tracks degrade almost immediately after $\beta = 1.2$, as the harsh mask chops into complex vocal harmonics, causing the SAR to plummet. Background instrumentals uniformly degrade as mask severity increases, proving that a single static Wiener mask cannot perfectly optimize both simultaneous extractions.

C. Detailed Genre Analysis at Optimal Tuning ($\beta = 1.4$)

The dataset was broken down by genre and evaluated at the established mathematical optimum of $\beta = 1.4$. As shown in Figure 4 and summarized in Table I, performance is heavily dependent on musical arrangement.

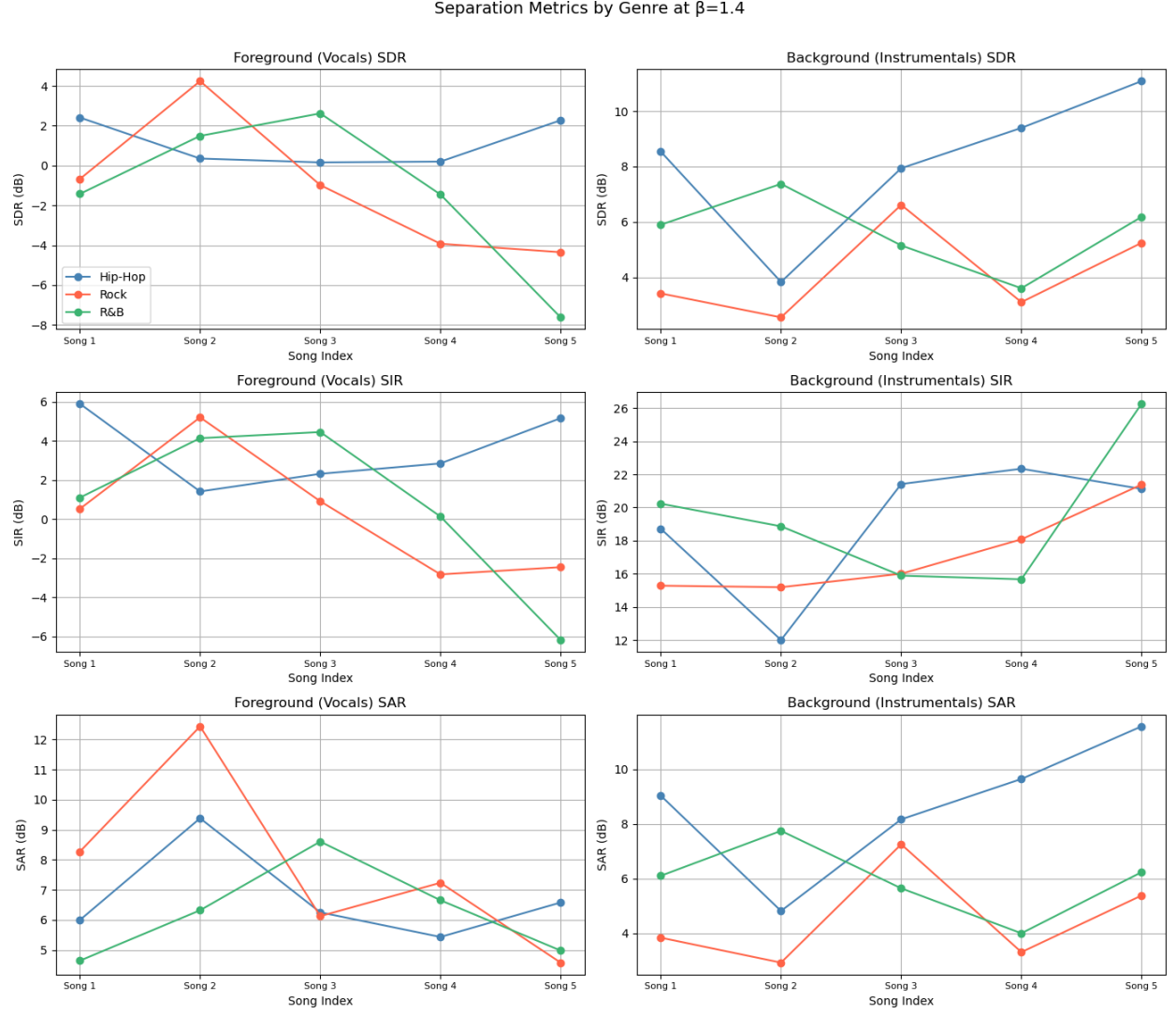


Fig. 4. BSS Eval metrics (SDR, SIR, SAR) across 5 songs per genre, evaluated at $\beta = 1.4$.

Genre	Vocals SDR (dB)	Vocals SIR (dB)	Vocals SAR (dB)	Inst. SDR (dB)	Inst. SIR (dB)	Inst. SAR (dB)
Hip-Hop	1.10	3.52	6.72	8.50	18.20	8.66
R&B	-1.25	1.05	6.24	5.60	19.30	5.94
Rock	-1.04	0.45	7.70	4.10	16.50	4.54

TABLE I

SUMMARY OF PER-GENRE AVERAGE EVALUATION METRICS AT OPTIMAL TUNING ($\beta = 1.4$).

- **Hip-Hop (Ideal Use-Case):** Consistently outperformed other genres. Because Hip-Hop

instrumentals are built on strictly quantized, perfectly cyclical digital drum loops, the median filter easily captures the model.

- **Rock (Worst-Case Scenario):** Live drummers naturally drift in tempo, blurring the auto-correlation peaks. Furthermore, Rock heavily features lead electric guitars; because guitar solos are non-repeating, REPET falsely classifies them as "vocals", creating massive track bleed.
- **R&B (High Volatility):** R&B often uses "soul samples" (sampled vocals used as part of the repeating beat). When a vocal loops, REPET correctly identifies it as repeating and traps it in the instrumental track, severely damaging isolation metrics.

Foreground Score Degradation and HPF Trade-offs:

The Foreground SDR is significantly lower than the Background SDR. Because the foreground is calculated via subtraction ($V_{foreground} = V_{mix} - M$), any instrumental frequencies that the median filter fails to capture inevitably "bleed" into the vocal track. To counteract this interference, a 300 Hz High-Pass Filter (HPF) was applied to the final vocal estimate. While this successfully stripped out the low-end kick drum bleed, the BSS Eval suite mathematically interprets this missing low-end frequency data as algorithmic distortion, inherently depressing the vocal SDR scores in exchange for a cleaner subjective listening experience.

Conclusion (Karaoke Application): While the statistical approach struggles with live instrumentation, *BeatPeeler* is highly effective for modern, quantized music. Specifically for the karaoke enthusiasts targeted in the introduction, the algorithm excels; the background instrumental retains high fidelity and structure, perfect for live sing-alongs even if the extracted vocal track contains algorithmic artifacts.

VI. SUGGESTIONS FOR EXTENSIONS

While the MVP successfully demonstrated on-device source separation, several extensions would improve the app:

- 1) **Full C++ Port of All DSP:** While multithreading for the median filter was successfully implemented in native C++ via JNI, the outer buffer management and File I/O still rely on a bridge. Porting the entirety of the pipeline strictly to native code would further optimize memory usage.

- 2) **Stereo Processing:** The current implementation downmixes audio to mono. Expanding the matrix math to support dual-channel arrays would preserve the stereo image of the separated tracks.
- 3) **DSP-ML Hybridization:** To overcome the mathematical limits of the Wiener mask, future iterations could use the REPET median filter to generate background features, but feed those features into a lightweight, on-device Convolutional Neural Network (CNN) to calculate the final mask, drastically improving the SAR metric.

VII. SOFTWARE AND HARDWARE DOCUMENTATION

A. Program Structure and File Responsibilities

1) Frontend & UI Management (Java):

- **MainActivity.java:** Serves as the Android entry point. It manages all user-facing interactions, Android `Intents` for file selection, and OS-level permissions. It also handles the UI threading, spawning a background worker thread for the C++ DSP functions to prevent "App Not Responding" (ANR) crashes while keeping the progress bar responsive.

2) Native Audio Engine & JNI Bridge (C++):

- **audio_main.cpp & audio_player.cpp:** These files manage the underlying Android native audio engine. They configure the OpenSL ES audio buffer queues and interface directly with the device's audio hardware to ensure low-latency rendering.
- **ece420_main.cpp:** This file acts as the translation layer (JNI Bridge). It decodes the input .wav file into a standard C++ `std::vector<float>`, routes the data to the DSP algorithm, and manages the `reportProgress` JNI callback to update the Java UI.

3) Core DSP Engine & Libraries (C++):

- **repet_separation.cpp:** The core mathematical pipeline. It executes the fundamental STFT, period detection, median filtering, and soft Wiener mask ($\beta = 1.4$) calculations. It implements a custom `parallelFor` wrapper to distribute median matrix calculations across the tablet's available CPU cores.
- **KissFFT Library:** A highly efficient, lightweight 1D Fast Fourier Transform library written in C. Because Android lacks a native standard library for complex frequency-domain mathematics, `KissFFT` is utilized heavily throughout the DSP pipeline to compute both the forward STFTs, the Autocorrelation, and the final Inverse STFT (ISTFT) reconstruction.

B. File Format and Integrity Constraints

To align with professional music production standards, the application strictly processes and exports uncompressed `.wav` files. While compressed formats like `.mp3` save storage space, their lossy compression algorithms inherently introduce phase distortion and frequency masking. Because REPET relies heavily on precise phase alignment for ISTFT reconstruction, utilizing compressed audio would fundamentally corrupt the mathematical integrity of the separation.

C. Logic Diagram (Data Flow)

- 1) **Input State:** User selects a `.wav` file via `MainActivity.java` and configures the Window Size and Overlap parameters.
- 2) **JNI Handoff:** Java spawns a background thread and calls `splitAudioFile`, passing the file paths to `ece420_main.cpp`.
- 3) **Memory Allocation:** `ece420_main.cpp` reads the `.wav` into a float vector and passes it to `repet_separation.cpp`.
- 4) **DSP Loop (`repet_separation.cpp`):**
 - Slice a 10-second window from the float vector.
 - Distribute `KissFFT` STFT computations across available CPU cores via `parallelFor`.
 - Calculate Autocorrelation to find the repeating period p .
 - Stack STFT segments of length p and compute the median background model.
 - Apply the Wiener mask and compute the ISTFT using `KissFFT`.
 - Apply the Hann window and overlap-add the chunk into the final output buffers.
 - Fire the JNI callback to update the Java progress bar.
- 5) **Output State:** `ece420_main.cpp` encodes the final background and foreground buffers into `.wav` files. The Java UI thread is notified.
- 6) **Playback:** `MainActivity.java` passes the separated audio files to the native audio engine (`audio_player.cpp`) for playback via OpenSL ES.

REFERENCES

- [1] Z. Rafii and B. Pardo, "Repeating pattern extraction technique (repet): A simple method for music/voice separation," *IEEE transactions on audio, speech, and language processing*, vol. 21, no. 1, pp. 73–84, 2012.